```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Animated Pendulum — The Mathematics</title>

<style>
 * {
  box-sizing: border-box;
 }

 body {
  margin: 0;
  padding: 16px;
  min-height: 100vh;
  background: #060b1c;
  color: #e7f3ff;
  font-family: Arial, sans-serif;
  display: flex;
  flex-direction: column;
  align-items: center;
 }

 main {
  width: min(100%, 900px);
```

```
}

canvas {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 20px;
  box-shadow: 0 20px 80px #0008;
}

nav {
  display: flex;
  gap: 12px;
  justify-content: center;
  align-items: center;
  flex-wrap: wrap;
  padding: 18px;
}

button,
select {
  padding: 12px 20px;
  border: 1px solid #3c6583;
  border-radius: 10px;
  background: #172a43;
  color: white;
```

```
  font: 700 17px Arial;
  cursor: pointer;
 }

 button:hover {
  background: #24516b;
 }

 p {
  text-align: center;
  color: #b1c9e5;
  font-size: 16px;
 }
</style>
</head>

<body>
<main>
 <canvas
  id="scene"
  width="900"
  height="1200"
  aria-label="Animated pendulum showing forces, energy and equations">
 </canvas>

 <nav>
```

```
    <button id="play">Pause</button>
    <button id="reset">Restart</button>

    <label>
     Speed
     <select id="speed">
      <option value="0.5">0.5×</option>
      <option value="1" selected>1×</option>
      <option value="2">2×</option>
     </select>
    </label>
   </nav>

   <p>
    Ideal pendulum: no friction, a point mass, and a massless rigid rod.
   </p>
  </main>

  <script>
  "use strict";

  const canvas = document.getElementById("scene");
  const ctx = canvas.getContext("2d");

  // ----------------------------------------
  // PHYSICS
```

```
// ----------------------------------------
// Units: metres, kilograms, seconds.
// Angles are calculated in radians.
//
// Full nonlinear equation:
// d²θ/dt² = -(g / L) × sin(θ)
//
// Angular velocity:
// ω = dθ/dt
//
// Kinetic energy:
// KE = ½ × m × (Lω)²
//
// Potential energy:
// PE = m × g × L × (1 - cos(θ))
// ----------------------------------------

const gravity = 9.81;
const length = 1.8;
const mass = 1;

const initialAngle = 38 * Math.PI / 180;
const stepSize = 1 / 240;

const totalEnergy =
  mass * gravity * length * (1 - Math.cos(initialAngle));
```

```
const smallAnglePeriod =
  2 * Math.PI * Math.sqrt(length / gravity);

const samples = [
  {
    angle: initialAngle,
    velocity: 0
  }
];

function angularAcceleration(angle) {
  return -(gravity / length) * Math.sin(angle);
}

// Fourth-order Runge–Kutta integration.
function integrate(state) {
  const h = stepSize;

  const k1Angle = state.velocity;
  const k1Velocity = angularAcceleration(state.angle);

  const k2Angle = state.velocity + k1Velocity * h / 2;
  const k2Velocity = angularAcceleration(
    state.angle + k1Angle * h / 2
  );
```

```
  const k3Angle = state.velocity + k2Velocity * h / 2;
  const k3Velocity = angularAcceleration(
   state.angle + k2Angle * h / 2
  );

  const k4Angle = state.velocity + k3Velocity * h;
  const k4Velocity = angularAcceleration(
   state.angle + k3Angle * h
  );

  return {
   angle: state.angle + h * (
    k1Angle + 2 * k2Angle + 2 * k3Angle + k4Angle
   ) / 6,

   velocity: state.velocity + h * (
    k1Velocity +
    2 * k2Velocity +
    2 * k3Velocity +
    k4Velocity
   ) / 6
  };
}

function getState(time) {
```

```
 const position = Math.max(0, time) / stepSize;
 const index = Math.floor(position);

 while (samples.length <= index + 1) {
  samples.push(integrate(samples[samples.length - 1]));
 }

 const first = samples[index];
 const second = samples[index + 1];
 const fraction = position - index;

 return {
  angle:
   first.angle +
   (second.angle - first.angle) * fraction,

  velocity:
   first.velocity +
   (second.velocity - first.velocity) * fraction
 };
}

// The first negative-to-positive crossing occurs at ¾ of a period.
function calculateFullPeriod() {
 let previous = getState(0.01);
```

```
  for (let time = 0.02; time < 5; time += 0.01) {
    const current = getState(time);

    if (previous.angle < 0 && current.angle >= 0) {
      const fraction =
        -previous.angle / (current.angle - previous.angle);

      const crossingTime = time - 0.01 + fraction * 0.01;

      return crossingTime * 4 / 3;
    }

    previous = current;
  }

  return NaN;
}

const fullPeriod = calculateFullPeriod();

// ------------------------------------------
// DRAWING HELPERS
// ------------------------------------------

const colors = {
  cyan: "#37e7ff",
```

```
  gold: "#ffce67",
  pink: "#ff65ad",
  mint: "#66ffd1",
  text: "#f2f7ff",
  muted: "#aec3de"
};

function drawText(
  content,
  x,
  y,
  size = 26,
  color = colors.text,
  weight = 400,
  align = "left"
) {
  ctx.font = `${weight} ${size}px Arial, sans-serif`;
  ctx.fillStyle = color;
  ctx.textAlign = align;
  ctx.fillText(content, x, y);
  ctx.textAlign = "left";
}

function drawLine(x1, y1, x2, y2, color, width = 2) {
  ctx.beginPath();
  ctx.moveTo(x1, y1);
```

```
  ctx.lineTo(x2, y2);
  ctx.strokeStyle = color;
  ctx.lineWidth = width;
  ctx.stroke();
}

function drawBox(
  x,
  y,
  width,
  height,
  fill = "#111f38",
  stroke = "#29415e",
  radius = 18
) {
  ctx.beginPath();
  ctx.roundRect(x, y, width, height, radius);

  ctx.fillStyle = fill;
  ctx.fill();

  if (stroke) {
    ctx.strokeStyle = stroke;
    ctx.lineWidth = 1;
    ctx.stroke();
  }
```

```
}

function drawArrow(x1, y1, x2, y2, color) {
 // Avoid drawing an arrowhead for a zero-length force.
 if (Math.hypot(x2 - x1, y2 - y1) < 0.5) {
  return;
 }

 drawLine(x1, y1, x2, y2, color, 4);

 const angle = Math.atan2(y2 - y1, x2 - x1);

 ctx.beginPath();
 ctx.moveTo(x2, y2);

 ctx.lineTo(
  x2 - 13 * Math.cos(angle - 0.45),
  y2 - 13 * Math.sin(angle - 0.45)
 );

 ctx.lineTo(
  x2 - 13 * Math.cos(angle + 0.45),
  y2 - 13 * Math.sin(angle + 0.45)
 );

 ctx.fillStyle = color;
```

```
  ctx.fill();
}

// ----------------------------------------
// ANIMATION SCENE
// ----------------------------------------

function render(time) {
  const state = getState(time);

  const angle = state.angle;
  const angularVelocity = state.velocity;

  const angleDegrees = angle * 180 / Math.PI;
  const linearVelocity = length * angularVelocity;

  const kineticEnergy =
    0.5 * mass * linearVelocity ** 2;

  const potentialEnergy =
    mass * gravity * length * (1 - Math.cos(angle));

  // Background.
  const background = ctx.createLinearGradient(0, 0, 900, 1200);
  background.addColorStop(0, "#071225");
  background.addColorStop(1, "#0e0d29");
```

```
ctx.fillStyle = background;
ctx.fillRect(0, 0, 900, 1200);

// Laboratory grid.
ctx.globalAlpha = 0.12;

for (let x = 0; x < 900; x += 40) {
  drawLine(x, 145, x, 650, "#5990b7");
}

for (let y = 160; y < 650; y += 40) {
  drawLine(0, y, 900, y, "#5990b7");
}

ctx.globalAlpha = 1;

// Headings.
drawText(
  "PHYSICS IN MOTION",
  40, 44, 21, colors.cyan, 700
);

drawText(
  "THE PENDULUM",
  40, 94, 48, colors.text, 800
```

```
);

drawText(
  "Gravity → motion → energy exchange",
  40, 133, 26, colors.muted
);

drawBox(686, 35, 174, 55, "#15273e");

drawText(
  `${time.toFixed(2)} s`,
  773, 72, 27, colors.mint, 700, "center"
);

// Convert physical position to screen coordinates.
const pivotX = 352;
const pivotY = 213;
const screenLength = 290;

const bobX = pivotX + screenLength * Math.sin(angle);
const bobY = pivotY + screenLength * Math.cos(angle);

// Swing arc.
ctx.setLineDash([5, 8]);
ctx.strokeStyle = "#5d7794";
ctx.lineWidth = 2;
```

```
ctx.beginPath();
ctx.arc(
  pivotX,
  pivotY,
  screenLength,
  Math.PI / 2 - initialAngle,
  Math.PI / 2 + initialAngle
);
ctx.stroke();
ctx.setLineDash([]);

// Equilibrium reference.
drawLine(
  pivotX, pivotY,
  pivotX, 548,
  "#5c7189", 1.5
);

// Fading motion trail.
for (let i = 50; i > 0; i--) {
  const previous = getState(
    Math.max(0, time - i * 0.012)
  );

  const x =
```

```
    pivotX + screenLength * Math.sin(previous.angle);

  const y =
    pivotY + screenLength * Math.cos(previous.angle);

  ctx.globalAlpha = (1 - i / 52) * 0.27;

  ctx.beginPath();
  ctx.arc(x, y, 12, 0, Math.PI * 2);
  ctx.fillStyle = colors.cyan;
  ctx.fill();
}

ctx.globalAlpha = 1;

// Angle measured from the downward vertical.
ctx.beginPath();
ctx.arc(
  pivotX,
  pivotY,
  70,
  Math.min(Math.PI / 2, Math.PI / 2 - angle),
  Math.max(Math.PI / 2, Math.PI / 2 - angle)
);
ctx.strokeStyle = colors.gold;
ctx.lineWidth = 4;
```

```
ctx.stroke();

drawText(
 "θ",
 pivotX + 25,
 pivotY + 91,
 29,
 colors.gold,
 700
);

// Rod.
ctx.shadowColor = colors.cyan;
ctx.shadowBlur = 12;

drawLine(
 pivotX, pivotY,
 bobX, bobY,
 "#b6e9ff", 7
);

ctx.shadowBlur = 0;

// Support and pivot.
drawBox(
 248, 180, 208, 20,
```

```
  "#263f5a", "#6587a1", 5
);

ctx.beginPath();
ctx.arc(pivotX, pivotY, 11, 0, Math.PI * 2);
ctx.fillStyle = "#effaff";
ctx.fill();

drawText(
  "L = 1.80 m",
  55, 243, 26, colors.muted, 700
);

drawText(
  "m = 1.00 kg",
  55, 278, 25, colors.muted
);

// Gravity acts vertically downward.
drawArrow(
  bobX, bobY,
  bobX, bobY + 88,
  colors.pink
);

// Tangential restoring force.
```

```
const tangentialForce =
  -mass * gravity * Math.sin(angle);

const forceX =
  tangentialForce * Math.cos(angle);

const forceY =
  -tangentialForce * Math.sin(angle);

drawArrow(
  bobX,
  bobY,
  bobX + forceX * 10,
  bobY + forceY * 10,
  colors.cyan
);

// Glowing pendulum bob.
ctx.shadowColor = colors.gold;
ctx.shadowBlur = 25;

const ball = ctx.createRadialGradient(
  bobX - 9, bobY - 11, 2,
  bobX, bobY, 24
);
```

```
ball.addColorStop(0, "#fffce4");
ball.addColorStop(0.4, "#ffe39a");
ball.addColorStop(1, "#f29a37");

ctx.beginPath();
ctx.arc(bobX, bobY, 23, 0, Math.PI * 2);
ctx.fillStyle = ball;
ctx.fill();

ctx.shadowBlur = 0;

drawText(
  "Gravity  mg ↓",
  50, 599, 23, colors.pink, 700
);

drawText(
  "Restoring force  F_t",
  50, 631, 23, colors.cyan, 700
);

// Energy panel.
drawBox(
  638, 170, 222, 410,
  "#101d34", "#2b3f5c"
);
```

```
drawText(
  "ENERGY",
  749, 205, 24, colors.text, 700, "center"
);

const bars = [
 {
   x: 674,
   value: potentialEnergy,
   color: colors.pink,
   label: "PE"
 },
 {
   x: 769,
   value: kineticEnergy,
   color: colors.cyan,
   label: "KE"
 }
];

for (const bar of bars) {
  drawBox(
    bar.x, 242, 53, 224,
    "#070f21", null, 8
  );
```

```
const height =
  224 * Math.min(
    1,
    Math.max(0, bar.value / totalEnergy)
  );

if (height > 0) {
  drawBox(
    bar.x,
    466 - height,
    53,
    height,
    bar.color,
    null,
    7
  );
}

drawText(
  bar.label,
  bar.x + 26,
  501,
  24,
  bar.color,
  700,
```

```
    "center"
  );

  drawText(
    `${bar.value.toFixed(2)} J`,
    bar.x + 26,
    535,
    21,
    colors.text,
    700,
    "center"
  );
}

drawText(
  `Total ${totalEnergy.toFixed(2)} J`,
  749, 566, 23, colors.gold, 700, "center"
);

// Live measurements.
const measurements = [
  {
    x: 40,
    label: "ANGLE θ",
    value: `${angleDegrees.toFixed(1)}°`,
    color: colors.gold
```

```
  },
  {
    x: 320,
    label: "SPEED |v|",
    value: `${Math.abs(linearVelocity).toFixed(2)} m/s`,
    color: colors.cyan
  },
  {
    x: 600,
    label: "ANGULAR ACCEL.",
    value: angularAcceleration(angle).toFixed(2),
    color: colors.pink
  }
];

for (const item of measurements) {
  drawBox(item.x, 661, 260, 93);

  drawText(
    item.label,
    item.x + 17,
    690,
    18,
    colors.muted,
    700
  );
```

```
  drawText(
    item.value,
    item.x + 17,
    732,
    32,
    item.color,
    700
  );
}

drawText(
  "Angular acceleration in rad/s²",
  858, 779, 18, colors.muted, 400, "right"
);

// Scrolling angle graph.
drawBox(40, 794, 820, 135, "#101d34");

drawText(
  "ANGLE OVER TIME",
  58, 822, 19, colors.muted, 700
);

const graphX = 70;
const graphY = 875;
```

```
const graphWidth = 756;
const graphScale = 39;

drawLine(
  graphX,
  graphY,
  graphX + graphWidth,
  graphY,
  "#52617c",
  1
);

ctx.beginPath();

let started = false;

for (let i = 0; i <= 360; i++) {
  const sampleTime = time - 6 + 6 * i / 360;

  if (sampleTime < 0) {
    continue;
  }

  const value =
    getState(sampleTime).angle / initialAngle;
```

```
  const x =
    graphX + i / 360 * graphWidth;

  const y =
    graphY - value * graphScale;

  if (!started) {
    ctx.moveTo(x, y);
    started = true;
  } else {
    ctx.lineTo(x, y);
  }
}

ctx.strokeStyle = colors.gold;
ctx.lineWidth = 3;
ctx.stroke();

ctx.beginPath();
ctx.arc(
  graphX + graphWidth,
  graphY - angle / initialAngle * graphScale,
  5,
  0,
  Math.PI * 2
);
```

```
ctx.fillStyle = colors.gold;
ctx.fill();

drawText(
  "0°",
  58, 882, 15, colors.muted
);

drawText(
  "last 6 seconds",
  833, 819, 17, colors.muted, 400, "right"
);

// Equations panel.
drawBox(
  40, 949, 820, 193,
  "#172039", "#374568"
);

drawText(
  "THE MATH BEHIND THE SWING",
  60, 982, 23, colors.gold, 700
);

drawText(
```

```
  "d²θ/dt² = −(g/L) sin θ   F_t = −mg sin θ",
  60, 1022, 25, colors.text, 700
);

drawText(
  "KE = ½m(Lω)²    PE = mgL(1 − cos θ)",
  60, 1063, 25, colors.cyan, 700
);

drawText(
  `T ≈ 2π√(L/g) = ${smallAnglePeriod.toFixed(
```